



ArozOS Cluster Technical White Paper

Many independent ArozOS nodes presented as one logical computer

Applies to	ArozOS v3.0.4 branch, cluster Phases 1 to 9
Audience	Developers, integrators and administrators
Sources	src/mod/cluster/README.md, TASKS.md, the Go packages under src/mod/cluster/, examples/clusters_jobs/
Date	2026-09-21

Contents

1. Executive summary
2. Design goals and non-goals
3. Architecture overview
4. Identifiers and addressing
5. ACN: the node-to-node protocol
6. Membership, joining and node states
7. Identity (single sign-on across nodes)
8. The metadata store (ACMS)
9. Storage and the `cluster:/` drive
10. Replication
11. The event bus
12. Jobs, map/reduce and the job contract
13. Scheduling
14. Developer interfaces: AGI, HTTP and Go
15. Worked examples
16. Minimum dependencies and requirements
17. Limits and maximum capabilities
18. Security model
19. Operations and administration
20. Developer guide: extending the cluster
21. Known constraints and pitfalls
22. Appendix A: endpoint reference
23. Appendix B: tunables and constants
24. Appendix C: glossary

1. Executive summary

The ArozOS cluster lets several independent ArozOS installations, which may run on different hardware, operating systems and CPU architectures, act as a single logical computer. A cluster provides four unified services:

- **One namespace.** A `cluster:/` drive appears in File Manager, WebDAV, media serving and every AGI library on every member. Files stay whole files inside ordinary folders that nodes contribute; a replicated metadata index says which nodes hold which copies.
- **One identity.** A user logs in on any member with the same credentials. Logins are forwarded to an *identity origin* node, with a replicated account directory as fallback when the origin is unreachable.
- **One compute pool.** AGI job scripts are submitted once and run on the best node for them, chosen by capability (for example `ffmpeg` or `nvidia`), data locality and load. Map/reduce spreads work across the nodes that already hold the data.
- **One API.** The AGI `cluster` library, an event bus with script hooks and a WebSocket feed, and admin HTTP endpoints hide node selection, routing, retries and replication from applications.

Every node keeps working as a normal standalone ArozOS when the cluster, or any other member, is unreachable. The whole feature ships inside the single ArozOS binary, adds no external service (no etcd, no ZooKeeper, no database server), requires no quorum, and works with as few as two nodes, including nodes behind NAT or Cloudflare.

2. Design goals and non-goals

2.1 Goals

1. **Heterogeneous nodes.** An x86 server, a Raspberry Pi and a Windows workstation must be able to cooperate. Nothing assumes a shared OS, architecture, installed tools or hardware.
2. **Autonomy.** A node is always a complete ArozOS. Losing the cluster degrades only the cluster features.
3. **Whole files.** Files are never split into blocks across nodes. Any copy is a normal file that can be read with ordinary tools, and the index can always be rebuilt by scanning the real files.
4. **Small clusters first.** Two-node clusters must work, including failover. This rules out majority-quorum consensus.
5. **Internet friendly.** Nodes may sit behind NAT, reverse proxies or Cloudflare. All transfers fit within Cloudflare request and idle limits.
6. **A stable application API.** Applications use AGI (the ArozOS JavaScript Gateway Interface) and never talk to node addresses.
7. **Portability.** Pure Go, no CGO requirement, no platform tools in shared code, build-tagged syscalls.

2.2 Non-goals

- A Hadoop/Ceph-style block file system or erasure coding.
- Strong linearizable consistency (no Raft or Paxos). Metadata converges last-writer-wins, serialised by a leader lease.
- Large-scale clusters. Heartbeats are a full mesh; the design targets households, labs and small offices, not hundreds of nodes.
- Running native binaries as jobs. Jobs are AGI scripts, because node architectures differ; native work is reached through AGI libraries (for example `ffmpeg`) on nodes that have the tool.

3. Architecture overview

3.1 Layers

```
+-----+
| Applications: File Manager, WebDAV, Photo, Movie, AGI apps |
+-----+
| AGI "cluster" library | cluster:/ drive | /system/cluster |
+-----+
| jobs (+ map/reduce) | events | replication | storage |
+-----+
| scheduling (one weighted scorer for all) |
+-----+
| metadata store (ACMS): leader lease + replicated log |
+-----+
| identity (AID) | membership (agent, gossip, health) |
+-----+
| ACN: Ed25519 signed HTTP, direct / tunnel / relay routing |
+-----+
| HTTP(S) on the normal ArozOS port, under /cluster/acn/* |
+-----+
```

3.2 Packages

All cluster code lives under `src/mod/cluster/` (module path `imuslab.com/arozos`).

Package	Role
<code>acn/</code>	ArozOS Cluster Node protocol: node keys, signed requests, replay protection, routing, WebSocket reverse tunnel
<code>capability/</code>	Portable detection of OS, arch, cores, RAM, tools and CPU flags; requirement matching; cross-platform DiskUsage
<code>membership/</code>	The cluster agent: create, join, leave, gossip of node records, heartbeats, node states, latency, cluster settings, admin endpoints
<code>identity/</code>	Forward authentication to the identity origin, account directory mirror, password write-back, signed user assertions
<code>metadata/</code>	Replicated namespace index (files, copies, volumes, policies, settings, jobs), leader lease, replicated log, snapshots, write-behind persistence
<code>storage/</code>	Contributed volumes, placement, chunked verified transfers, reads and writes for the drive, reconcile, copy primitives
<code>replication/</code>	Leader-side planner and per-node worker keeping every file at its policy's copy count
<code>events/</code>	Publish/subscribe with cluster-wide fan-out and de-duplication, AGI hooks, WebSocket feed
<code>jobs/</code>	Replicated job records, leader-side scheduler, per-node execution, leases, cancellation, map/reduce
<code>scheduling/</code>	The single placement scorer used by jobs, writes and replication
<code>wakeonlan/</code>	Wake-on-LAN packets for offline LAN neighbours

Related code outside the package:

File	Role
<code>src/cluster.go</code>	Core wiring (<code>ClusterInit</code> , <code>clusterStartAgent</code>), drive mount/unmount, AGI provider, event hooks, master-node resolver, shutdown

File	Role
src/cluster.jobs.go	Job executor adapter to the AGI gateway, job submit handler, sched/explain
src/cluster.fsinfo.go	Cluster tab of the File Manager properties dialog
src/mod/filesystem/abstractions/clusterfs/	The cluster:/ file system abstraction
src/mod/agi/agi.cluster.go	The AGI cluster library
src/main.router.go	Mounts /cluster/acn/* before the session check
src/web/SystemAO/cluster/	Cluster Settings, Cluster Info and Cluster Jobs pages
src/web/SystemAO/locale/cluster.json	UI and server message translations
examples/clusters_jobs/	Runnable job examples

3.3 Start-up order

ClusterInit (called from src/startup.go) runs clusterStartAgent unless -disable_cluster is set. Services are created in dependency order, and each is optional: if one fails to start, the services below it are skipped and the failure is logged under the Cluster title.

```
membership.NewManager      (system/cluster.db, system/cluster/node.key)
-> identity.New            (hooks authAgent.ForwardAuth)
-> metadata.New
    -> scheduling.New
    -> storage.New          (mounts cluster:/ via clusterSyncDrive)
        -> events.New      (file / replica / disk-full events)
        -> jobs.New        (user routes, "Cluster Jobs" page)
        -> AGI ClusterLibRegister()
        -> replication.New (nightly verify task)
```

AGI starts before the cluster, so the cluster library is registered late through AGIGateway.ClusterLibRegister() once the provider exists. ClusterShutdown closes the services in reverse order so heartbeats and tunnels stop cleanly.

4. Identifiers and addressing

Nothing in the cluster is addressed by a name. Names are operator-set and every fresh install is called `My ArOZ`, so two nodes can share a name. Only the IDs below are unique.

Thing	Identifier	Origin	Unique within
Cluster	<code>ClusterInfo.ID</code> (UUID v4)	<code>CreateCluster</code> , carried in join tokens	everywhere
Node	<code>NodeRecord.ID</code> = device UUID	<code>system/dev.uuid</code> or the <code>-uuid</code> flag	the cluster
Node identity	Ed25519 key pair	<code>system/cluster/node.key</code>	everywhere
Node name	<code>NodeRecord.Name</code>	<code>-hostname</code> , default <code>My ArOZ</code>	not unique
Drive on a node	<code>FileSystemHandler.UUID</code> (<code>user</code> , <code>s1</code> , ...)	node storage config	that node only
Volume	<code>Volume.ID</code> (UUID v4)	<code>AddVolume</code>	everywhere
Volume location	<code>NodeID</code> + <code>FshUUID</code> + <code>Subpath</code>	<code>AddVolume</code> (duplicates refused)	everywhere
File	<code>FileRecord.ID</code> (UUID v4), keyed by <code>Path</code>	first write of the path	the cluster
Copy	<code>VolumeID</code> (+ <code>NodeID</code>) in the record	placement / replication	per file
Job	<code>Job.ID</code> (UUID v4)	job submit	the cluster

Because a drive UUID only means something on one node, every physical location the cluster displays is written **node first**:

```
<node uuid>:<drive uuid>/<path on that drive>  
3f7a...c1:user/cluster/photos/a.jpg
```

Warning: cloned installs. Copying `system/dev.uuid` to a second host gives both the same node ID and their membership records merge into one. Delete `system/dev.uuid` and `system/cluster/` on a clone before it joins.

5. ACN: the node-to-node protocol

5.1 Signed requests

Every node owns an Ed25519 key pair (`acn.LoadOrCreateNodeKey`). The public key travels in the node's membership record. Every ACN request carries:

```
X-Aroz-Node:      <sender node UUID>
X-Aroz-Cluster:   <cluster UUID>
X-Aroz-Timestamp: <unix seconds>
X-Aroz-Nonce:     <random hex>
X-Aroz-Signature: base64(Ed25519(
                    METHOD \n target \n cluster \n node \n
                    ts \n nonce \n sha256(body)))
```

The receiver checks, in order: the sender is a known member, the cluster ID matches, the timestamp is within **plus or minus 5 minutes**, the nonce is unseen, and the signature verifies. Only after all checks pass is the nonce recorded, so junk requests cannot poison the replay cache. Signed bodies are capped at 64 MB.

ACN is mounted at `/cluster/acn/*` in `src/main.router.go` **before** the user-session check, because nodes authenticate with signatures rather than cookies. `cluster` is therefore a reserved subservice path.

5.2 Routing

Callers use `acn.Transport.Do / DoJSON(ctx, nodeID, method, path, in, out)` and never deal with addresses. The transport picks the route:

1. **Tunnel**: the peer has a live tunnel terminating on this node.
2. **Direct**: the peer advertises a URL (behind Cloudflare is fine).
3. **Relay**: the peer is tunnelled to another member `via`; the request goes to `https://<via>/cluster/acn/relay/<peer>/cluster/acn/<path>` and the tunnel host forwards it. The signature covers the *original* path, so the relay and the final node verify the same signature.

A (public URL)	B (public URL)	C (behind NAT)
---- direct HTTPS --->		
	<==== WebSocket =====	C tunnels to B
-- relay/C/... -----> ==== frame =====>		A reaches C via B

5.3 Reverse tunnel

A node without a public URL opens one WebSocket to a reachable member (`GET /cluster/acn/tunnel`, signed). Requests are multiplexed as binary frames `[kind][header length][header JSON][body]`, executed against the node's own ACN handler and answered on the same socket.

- Both ends ping every **30 s** (read timeout 90 s), so Cloudflare's 100 s idle limit never trips.
- The client reconnects with backoff and re-picks a host: the admin's preferred host, otherwise the healthiest reachable member.
- Frames are capped at **16 MB**; file transfers use 4 MiB chunks, well under Cloudflare's 100 MB request limit.
- A NAT-only node still appears online everywhere, because its tunnel host learns its `LastSeen` and gossips it.

6. Membership, joining and node states

6.1 Node records and gossip

Each node owns one `NodeRecord` (ID, name, public key, advertised URL, tunnel host, version, capabilities, admin state) with an `Updated` version stamp. Records travel with every heartbeat and merge **last-writer-wins** per record, so membership converges without a master.

- A node is authoritative for its own record; others may only change its admin state or remove it.
- Removed nodes remain as **tombstones for 7 days** so the removal propagates to members that were offline.
- Heartbeats go to every peer every **15 s** (full mesh).
- Versions come from `nextVersion(prev)`, strictly greater than the previous value; one millisecond is not unique, so never compare with `>=`.

6.2 Computed node states

Liveness is computed locally, not gossiped:

State	Meaning
ONLINE	heard from within 45 s
DEGRADED	online, but CPU or RAM at or above 97 %, or disk at or below 2 % free
UNKNOWN	silent for 45 s to 3 min
OFFLINE	silent for more than 3 min
MAINTENANCE / DRAINING	set by an admin, overrides the above

6.3 Capability manifests

Every node publishes a `capability.Manifest` in its record:

```
type Manifest struct {
    OS, Arch    string          // runtime.GOOS / GOARCH
    CPUCores    int
    TotalRAM    int64           // bytes, 0 when unknown
    Hostname    string
    Features    map[string]bool
    GoVersion   string
    DetectedAt  int64
}
```

Detected features:

Feature	Detected by
ffmpeg, ffprobe, docker, git, python3, node	executable found on PATH
nvidia (and then cuda, gpu)	nvidia-smi found on PATH
avx, avx2, avx512	golang.org/x/sys/cpu (x86)
neon	golang.org/x/sys/cpu (ARM64 ASIMD)
64bit	architecture name ends in 64

Jobs express `capability.Requirements ( features, minRam, minCores, os, arch)`; `Manifest.Satisfies` returns the first unmet requirement as a human-readable reason, which is what a queued job shows.

6.4 Creating and joining

1. On a member **that has an advertised URL**, an admin generates a join token in System Settings > Cluster Settings. The token is `aroz-join:<base64 JSON>` and embeds the cluster ID, the issuer's URL and a secret; only the secret's SHA-256 is stored. Tokens expire (default 24 hours) and can be listed and revoked.
2. The admin pastes the token on the new node. It POSTs its record to the issuer's `/cluster/acn/join`, receives the cluster info and member list, and starts heartbeating. A node without a URL tunnels to the issuer immediately.
3. The issuer pushes the new member list to everyone else.

Leaving (`/system/cluster/leave`) notifies peers, wipes every table registered with

`membership.RegisterClusterTable` and unmounts `cluster:/`. An admin can also remove a node, which sends it `/cluster/acn/evict`.

7. Identity (single sign-on across nodes)

One member can be made the **identity origin** (System Settings > Cluster > Identity). The choice is a cluster-wide setting replicated by gossip (`ClusterInfo.IdentityOrigin`, last-writer-wins on `SettingsVersion`).

On every other member the auth agent's `ForwardAuth` hook runs before the local password table:

1. **Forward auth.** The member sends the username and the SHA-512 hash of the typed password to the origin (`POST /cluster/acn/auth/verify`, signed). The origin compares in constant time and answers with the user's group names. The clear-text password never leaves the member.
2. **Mirror.** On success the member creates or updates the account locally (hash plus the groups that exist on that node, matched by name) and opens a normal local session.
3. **Fallback.** If the origin is *unreachable*, the local table decides. Members pull the origin's directory (`GET /cluster/acn/auth/directory`) at boot, every 5 minutes and when the origin setting changes. If the origin is reachable and says the password is wrong, the login is rejected with no fallback.
4. **Write-back.** Password changes of mirrored accounts are forwarded to the origin (`POST /cluster/acn/auth/setpassword`). Core code that changes a password must call `clusterNotifyPasswordChanged` after writing the hash.

Rules: only mirrored accounts are ever touched; a pre-existing local account with the same name is left alone ("local only"); accounts whose groups do not exist on a member are skipped and listed. **Permission groups are node-specific**, so create the same groups on every node.

7.1 Signed user assertions

Node A can act on node B for a logged-in user without contacting the origin:

```
token = base64url(payload) "." base64url(Ed25519 signature)
header = X-Aroz-User: <token>
```

`identity.Issue(username, ttl)` creates it (default lifetime 5 minutes), `Attach(req, username)` sets the header and `FromRequest(r)` verifies it against the issuer's published node key, returning `*identity.Assertion{User, Groups, Issuer}`.

8. The metadata store (ACMS)

The ArozOS Cluster Metadata Store is the replicated index of the namespace. File contents never pass through it.

8.1 Record kinds

Kind	Type	Key fields
file	FileRecord	ID, Path, IsDir, Size, ModTime, Checksum (SHA-256), Owner, Primary, Locations[], Replicas, Removed, Version
volume	Volume	ID, NodeID, Name, FshUUID, Subpath, Capacity, Free, ReadOnly, Evacuating, LowSpace, Removed
policy	Policy	Folder (top level, e.g. /photos), Replicas
setting	Setting	Key (e.g. scheduling.weights), Value (JSON)
job	Job	ID, Owner, Node, Status, Body (full job record)

A copy of a file is a `Location{VolumeID, NodeID, State, Checksum, Updated}` with a state of `pending`, `writing`, `committed`, `verified`, `stale` or `failed`. A copy is *healthy* when `committed` or `verified`.

8.2 Consistency model

Every record carries a `Version` from `membership.NextVersion` and merges **last-writer-wins** on every node. There is deliberately no Raft: the store converges without a majority, and correctness can always be restored by rescanning the real files (storage reconcile).

- **Leader lease** (`lease.go`). The eligible member that joined first (ties by ID) claims a **30 s** lease with a higher term when no live lease exists and renews it every **10 s**. Every member accepts a lease with a higher term. No quorum is needed, so a two-node cluster fails over. `IsLeader()` / `Leader()` serialise decisions: placement, replication planning and job scheduling.
- **Replicated log** (`log.go`). `Submit(kind, record)` applies locally at once; the leader assigns a sequence number and pushes the entry to online peers (`meta/append`). Followers submit through the leader (`meta/submit`) or queue in `meta_pending` until one is reachable. Gaps are filled with `meta/log?after=N`; a compacted range, a new term or a fresh follower (`applied == 0`) triggers a full `meta/snapshot`.
- **Write-behind persistence** (`persist.go`). Reads come from memory. A change updates memory under the store lock and queues a disk write; writes are coalesced per key and committed in one `database.WriteBatch` transaction every **50 ms**. A crash loses at most one flush interval of this node's disk copy, which catch-up restores like any other gap.

Rule. Never add a synchronous disk write inside the store lock. A write burst would starve lease renewal and the leader would lose its lease.

Read API: `Stat`, `ListDir`, `Glob`, `Volumes`, `PolicyFor`. Write API: `Submit(kind, record)`. New tables go through `membership.RegisterClusterTable` so they are wiped on leave.

9. Storage and the `cluster:/` drive

9.1 Volumes

An admin contributes a folder of a local drive as a **volume** (`storage/volume/add`, e.g. `user:/cluster`). Files in the namespace are ordinary files inside those folders. A rescan (`storage/rescan`, and automatically every 30 minutes) adopts files placed there by other means and flags copies that vanished or changed as `stale`. Volume capacity and free space are refreshed every 60 s.

9.2 The drive

When a node is in a cluster **and the cluster has at least one volume**, the core mounts a `Cluster` drive (`cluster:/`, public hierarchy, buffered) into the base storage pool (`clusterSyncDrive` in `src/cluster.go`, re-checked on every membership change and volume record change). File Manager, WebDAV, media serving and AGI `filelib` then use it like any other drive. Without a cluster or a volume the drive is unmounted, so nightly tasks and scanners do not see it.

9.3 Write path

```
client write
-> spool locally while hashing (SHA-256)
-> ask leader for placement (scheduling scorer; local volume first,
    then existing primary, then most free space)
-> copy bytes: local rename-in-place, or chunked upload to the target
-> publish FileRecord with a committed Location
```

The namespace never shows a half-written file: the record is published only after the bytes are in place. Replication then brings the file up to its policy's copy count.

9.4 Read path

A read opens a local healthy copy when there is one; otherwise it streams from an online node through the chunked protocol while verifying the checksum.

9.5 Chunked transfer protocol

Endpoint	Purpose
<code>store/place</code>	ask the leader where a new file should go
<code>store/begin</code>	open an upload session (resumable)
<code>store/chunk</code>	one signed chunk, per-chunk SHA-256
<code>store/commit</code>	whole-file SHA-256 check, rename <code>.part-<session></code> into place
<code>store/abort</code>	discard a session
<code>store/read</code>	ranged, chunked read
<code>store/stat</code> , <code>store/list</code> , <code>store/checksum</code>	inspect copies
<code>store/mkdir</code> , <code>store/rename</code> , <code>store/delete</code>	namespace operations on a volume

Chunks are **4 MiB** (maximum 8 MiB) and sessions expire after 30 minutes of inactivity.

9.6 Space guard and maintenance

- **Nearly full volumes.** Below **5 %** free a volume becomes read only and `LowSpace`, stops taking new files and emits a `node.diskfull` event; it recovers above **7 %**. The cluster setting `storage.autoReadOnly` (Cluster Settings > Storage) turns this off. Read-only set by an admin is never touched by the guard.
- **Nightly maintenance on the master node only.** Every member sees the same files on `cluster:/`, so trash expiry and version-history cleanup of a shared drive run only on the **master node** (the metadata leader): register such tasks with `nightly.TaskOption{MasterNodeOnly: true}`; `nightlyShouldMaintainFsh` answers per file system handler. A standalone host is its own master, so nothing changes for a single node.
- **Where is my file?** The abstraction implements `arozfs.StorageInfoProvider`, so the File Manager properties dialog shows a Cluster tab with every copy, its node and volume (node-first path), state, replica policy and checksum (`/system/file_system/getStorageInfo -> src/cluster.fsinfo.go`).

10. Replication

The replication manager keeps every file at the copy count its folder policy (or the record's own `Replicas`) asks for, on **different nodes**.

10.1 Planner (metadata leader, every 60 s and on demand)

Situation	Action
fewer healthy copies than wanted	one pull task to a node without a copy; an existing stale copy is repaired in place
more copies than wanted	drop the copy on the fullest non-primary volume, only when the rest are healthy
copy on an <i>evacuating</i> volume or a <code>DRAINING</code> node	re-create elsewhere, then drop
node <code>OFFLINE</code> for more than 10 minutes	mark its healthy copies <code>stale</code> (never deleted; reconcile restores them when it returns)

Limits: 4 tasks in flight per node, 16 in total, 4 workers per node. A file that fails backs off exponentially and is left alone for an hour after 5 attempts. Tasks are ephemeral (in memory with a 50-entry history): a new leader simply plans again within one interval.

10.2 Worker

The target node pulls the bytes with the chunked protocol (`storage.Service.PullCopy`), verifies the checksum, publishes a `verified` location and reports to the leader, renewing a **30 s** task lease every **10 s**.

10.3 Verification and evacuation

- A nightly pass re-checksums up to **1 GB** of this node's copies, least recently checked first, and downgrades mismatches to `stale`. "Verify this node" runs it on demand (`repl/verify`).
- `storage/volume/evacuate` marks a volume read only; the planner moves everything off it and the volume is retired when nothing references it. `volume/remove` refuses a volume that holds the only copy of a file.

Replica counts range from **1 to 16** (policy) and 0 to 16 per file, where 0 means "use the folder policy".

11. The event bus

`events.Bus` provides publish/subscribe with cluster-wide fan-out (`/cluster/acn/events/publish`, batched every 200 ms) and de-duplication by event ID over a 10-minute window.

```
type Event struct {
    ID, Type, Node, Path, FileID, User string
    Time int64
    Data json.RawMessage
}
```

Event type	Raised when
<code>file.created</code> , <code>file.removed</code> , <code>file.renamed</code>	the storage layer writes, removes or renames a file (<code>renamed</code> carries <code>{"from": ...}</code>)
<code>replica.verified</code> , <code>replica.stale</code>	a copy is verified or downgraded (<code>{"volume": ...}</code>)
<code>node.joined</code> , <code>node.left</code> , <code>node.online</code> , <code>node.offline</code>	computed locally by every node from membership
<code>node.diskfull</code>	a volume crosses the 5 % free guard
<code>job.completed</code> , <code>job.failed</code>	a job reaches a terminal state
<code>app.*</code>	custom events from <code>cluster.emit</code> (payload under 64 KB)

Consumers:

- **AGI hooks.** `cluster.on(types, scriptVpath)` registers a persistent hook (table `event_hooks`) that runs the script as its owner, with at most 4 hook runs at a time per node. The script reads the event with `postPara("event")` (JSON). Types accept wildcards such as `file.*`.
- **WebSocket feed.** Any logged-in user can subscribe at `/system/cluster/events/ws`.
- **Go.** `bus.Publish(events.Event{...})` from core code.

12. Jobs, map/reduce and the job contract

12.1 Model

A job is an AGI script that defines `run(job)`. The script **source is captured at submit time** and stored in one replicated record (`jobs.Record{Spec, State}`, metadata kind `job`), so any node answers status queries locally and a new leader resumes scheduling. Because it is JavaScript executed by the Otto VM inside ArozOS, the same job runs on x86, ARM, Windows, Linux or macOS nodes.

```
submit (any node) -> record queued -> leader schedules every 3 s
-> jobs/run to chosen node -> VM runs as the owner
-> lease renew every 10 s -> jobs/done -> record succeeded/failed
```

Statuses: `queued`, `scheduled`, `running`, `succeeded`, `failed`, `cancelled`. Kinds: `run`, `map`, `reduce`, `mapreduce`.

12.2 The job contract

```
function setup(ctx) {          // optional, runs once before run(); ctx = {node}
}

function run(job) { // job = {id, name, kind, args, inputs, node, owner}
  job.log("text");           // appended to the job log (last 200 lines)
  job.progress(0.5);         // 0..1
  if (job.cancelled()) {}    // poll for cancellation
  job.abortIfCancelled();    // throws when the job was cancelled
  return { ok: true };       // JSON-serialisable; becomes the job output
}
```

- Normal AGI libraries (`filelib`, `imagelib`, `ffmpeg`, ...) work inside a job and see `cluster:/`.
- **Use absolute paths** (`cluster:/...`, `user:/...`). A job has no script folder of its own, so relative paths are not resolved.
- The job runs **as the submitting user** on the chosen node. The owner must have an account there; with an identity origin configured, accounts are mirrored automatically.

12.3 Spec fields

Field (AGI / HTTP form)	Meaning	Default
<code>name</code>	display name	script name
<code>script</code>	vpath of the <code>.agi</code> file (relative to the launcher in AGI)	required
<code>args</code>	JSON passed as <code>job.args</code>	none
<code>inputs</code>	files the job reads; nodes holding them are preferred	none
<code>features</code>	required capabilities, e.g. <code>["ffmpeg"]</code>	none
<code>nodes</code>	allowed node IDs; others are ineligible	any node
<code>timeout</code>	seconds	3600
<code>priority</code> (HTTP)	higher runs first	0
<code>cores</code> (HTTP)	minimum CPU cores	none
<code>dataset</code> , <code>partitionMax</code> / <code>partition</code>	map/reduce only	50 files

`MaxAttempts` defaults to 3: when a running node goes silent the leader requeues the job, and fails it after the last attempt.

12.4 Execution guarantees

- **Parallelism.** At most `runtime.NumCPU()` jobs run per node; the rest wait.
- **Dispatch once.** Scheduling passes are serialised, and a node accepts each job once even if a hand-over is repeated. Temporary refusals (a tunnel still connecting, the leader not yet known) are retried. Only "the owner has no account here" or "this node cannot run jobs" block that node from that job for good (`State.Blocked`).
- **Timeouts and cancellation.** `agi.Gateway.ExecuteJobScript` hands back a stopper; a timeout or `cancel` interrupts the VM within about a second.
- **Pinning.** `nodes: [id]` sends a job to a specific member; one pinned job per member runs something everywhere (see `hello_world`).
- **Retention.** Finished records are kept for 72 hours.
- **Queued with a reason.** A job no node can take stays queued, and `state.reason` explains why (for example no online node has `ffmpeg`).

12.5 Map/reduce

A job with a `dataset` glob becomes a map/reduce parent. The leader coordinates it and never assigns the parent to a node.

1. The glob is expanded over the namespace (`metadata.Glob: *` within a folder, `**` across folders, `?` one character).
2. Files are grouped by the node that already holds a healthy copy and split into partitions of `partitionMax` files (default 50); one **map** child is submitted per partition, so map work runs where the bytes are.
3. When all map children succeed, emitted pairs are grouped by key and a single **reduce** child is submitted; its output becomes the parent's.
4. A failed child fails the parent and cancels the rest. A dataset with an unreachable file fails and names it; an empty dataset succeeds with an empty result. Grouped map output is capped at **8 MB**.

```
function mapper(files, emit) {          // files = paths of this partition
  files.forEach(function (f) { emit(f.split(".").pop(), 1); });
}
function reducer(key, values) {         // called once per key
  return values.length;
}
```

Mapper paths are namespace paths such as `/photos/a.jpg`; prefix them with `cluster:` to open them with `filelib`. Children inherit the parent's `nodes` restriction.

13. Scheduling

Three layers must answer "which node?": the job scheduler, write placement and the replication planner. They all call one scorer in `scheduling/`, which scores each candidate from 0 to 1:

Factor	Default weight	Measured as
data locality	0.45	share of the input bytes already on the node
free CPU	0.20	$1 - \text{cpuUsage}$ from the health report
free memory	0.10	$1 - \text{ramUsed} / \text{ramTotal}$
free disk	0.05	largest free fraction among the node's volumes
queue depth	0.10	penalty, saturating at 8 queued items
network distance	0.05	penalty from heartbeat RTT, saturating at 1000 ms
health	0.05	penalty when <code>DEGRADED</code>
wanted features	0.10	optional capabilities a job prefers
site diversity	0.05	distance of a new copy from existing copies

- **Weights** are one replicated setting (`scheduling.weights`), each 0 to 1, edited on the Cluster Settings page or through `sched/weights` (`reset=true` restores defaults). Missing weights read back as the shipped defaults, so old records stay valid.
- **Eligibility.** The caller supplies the hard rule (capability match, usable state, a volume with room). The scorer holds `DEGRADED` nodes back whenever a healthy eligible node exists, sorts ineligible nodes last and keeps their reason, so an empty result is explained rather than silent.
- **Latency** is the heartbeat round trip smoothed with an EWMA (alpha 0.3); the local node reports 0 and a never-heard peer reports -1.
- **Site diversity** is only computed with three or more nodes and more than one possible target. The planner pulls each member's latency vector over `GET /cluster/acn/latency` and caches the matrix for two minutes; a candidate's diversity is its distance to the nearest existing copy, as a fraction of one second.
- **Explanations.** Every score carries its factors; `/system/cluster/sched/explain?job=<id>` re-runs the ranking for a job and returns every candidate with its factors and reasons.

14. Developer interfaces

14.1 AGI `cluster` library

Load with `requirelib("cluster")`. File access on `cluster:/` needs no special library: `filelib` and every other library see the mounted drive.

Function	Returns	Notes
<code>cluster.inCluster()</code>	bool	
<code>cluster.self()</code>	object	this node: id, name, state, capabilities, health
<code>cluster.nodes()</code>	array	every member with state, platform, load
<code>cluster.status()</code>	object	cluster info, identity origin, leader, volumes, storage, replication
<code>cluster.stat(path)</code>	object	record with size, checksum, <code>locations[]</code> ; needs read permission
<code>cluster.list(path)</code>	array	records of a directory's children
<code>cluster.setReplicas(path, n)</code>	bool	admin; 0 to 16, 0 = folder policy
<code>cluster.policy(folder, n)</code>	bool	admin; top-level folder such as <code>/photos</code>
<code>cluster.on(types, scriptVpath)</code>	string	hook id; string or array, <code>file.*</code> wildcards
<code>cluster.off(hookId) / cluster.hooks()</code>	bool / array	
<code>cluster.emit(type, data)</code>	bool	<code>app.</code> prefix added; payload under 64 KB
<code>cluster.jobs.submit(spec)</code>	string	job id
<code>cluster.jobs.mapreduce(spec)</code>	string	{name, script, dataset, partitionMax, features, timeout}
<code>cluster.jobs.status(id)</code>	object	{spec, state}
<code>cluster.jobs.list()</code>	array	own jobs
<code>cluster.jobs.cancel(id)</code>	bool	
<code>cluster.jobs.wait(id, timeoutSec)</code>	object	blocks in Go; throws if not finished in time

Users see and cancel only their own jobs; administrators see all.

14.2 HTTP endpoints for browsers

All go through the permission router (`prout.NewModuleRouter`):

- **Admin only** (`System Setting` module, admin): `/system/cluster/*` status, create, join, leave, tokens, nodes, identity, metadata, storage, replication and scheduling. See Appendix A.
- **Logged-in users** (`Tasks Scheduler` module permission): `/system/cluster/jobs/{submit,status,get,cancel}`.
- **Logged-in users**: `/system/cluster/events/ws` (WebSocket) and `/system/cluster/events/hooks`.

14.3 Go building blocks

Need	Use
Signed request to a member	<code>clusterManager.Transport().DoJSON(ctx, nodeID, method, path, in, out)</code>
New signed node endpoint	<code>clusterManager.Server().HandleFunc(acn.BasePath+"/x", func(w, r, sender *acn.SignedIdentity, body []byte))</code>

Need	Use
Members and their state	<code>clusterManager.NodeViews()</code> (<code>State</code> , <code>Local</code> , <code>Capabilities</code> , <code>Health</code> , <code>LatencyMs</code> , ...)
Own IDs	<code>clusterManager.NodeID()</code> , <code>Cluster()</code> , <code>InCluster()</code>
Cluster-scoped storage	<code>clusterManager.DB()</code> (<code>system/cluster.db</code>) + <code>membership.RegisterClusterTable(name)</code>
React to setting changes	<code>chain OnClusterChange / OnMembershipChange</code> (keep the previous callback)
Namespace	<code>clusterMetadata.Submit</code> , <code>Stat</code> , <code>ListDir</code> , <code>Glob</code> , <code>Volumes</code> , <code>PolicyFor</code> , <code>IsLeader</code>
Placement	<code>clusterScheduling</code> (<code>Rank</code> , <code>Best</code>)
Capabilities	<code>capability.Manifest.Satisfies(capability.Requirements)</code> , <code>capability.DiskUsage(path)</code>
Cross-node user	<code>clusterIdentity.Issue</code> , <code>Attach</code> , <code>FromRequest</code>
Events	<code>clusterEvents.Publish(events.Event{...})</code>
Master-only nightly work	<code>nightly.TaskOption{MasterNodeOnly: true}</code>

15. Worked examples

The folder `examples/clusters_jobs/` contains three runnable pairs. Each has a **launcher** (`<name>.agi`, run like any AGI script, for example from the Serverless tool, Code Studio or `/system/ajgi/interface?script=<path>`) and a **job** (`<name>.job.agi`, captured at submit time). Keep both files in the same folder: the launcher refers to its job by a relative path.

Example	Shows
<code>hello_world/</code>	one job pinned to every usable node with <code>nodes</code> ; each replies with its name, platform, capabilities and health
<code>list_files/</code>	one job walks <code>cluster:/</code> with <code>filelib</code> and totals files per extension
<code>ffmpeg_convert/</code>	converts <code>cluster:/demo.mp4</code> to WebM on a node with <code>ffmpeg</code> , preferring one that already holds the video

15.1 Run on every node (launcher)

```
requirelib("cluster");

var nodes = cluster.nodes(), ids = [];
for (var i = 0; i < nodes.length; i++) {
  var n = nodes[i];
  if (n.state !== "ONLINE" && n.state !== "DEGRADED") continue;
  ids.push(cluster.jobs.submit({
    name: "Hello World on " + n.name,
    script: "hello_world.job.agi",
    nodes: [n.id],           // this node and nowhere else
    timeout: 60
  }));
}
var replies = [];
for (var j = 0; j < ids.length; j++) {
  var rec;
  try { rec = cluster.jobs.wait(ids[j], 60); }
  catch (e) { rec = cluster.jobs.status(ids[j]); }
  if (rec.state.status === "succeeded") replies.push(rec.state.output);
}
sendJSONResp(JSON.stringify(replies));
```

15.2 The matching job

```
requirelib("cluster");

function run(job) {
  var me = cluster.self();
  job.log("Hello World from " + me.name);
  return {
    message: "Hello World from " + me.name,
    os: me.capabilities.os,
    arch: me.capabilities.arch,
    cpuUsage: me.health.cpuUsage,
    runAs: job.owner
  };
}
```

15.3 Capability and locality (ffmpeg)

```
requirelib("cluster");
var id = cluster.jobs.submit({
  name: "Convert demo.mp4 to WEBM",
  script: "ffmpeg_convert.job.agi",
  args: { input: "cluster:/demo.mp4", output: "cluster:/demo.webm" },
  inputs: ["cluster:/demo.mp4"], // prefer a node holding the video
  features: ["ffmpeg"],          // only nodes with ffmpeg
  timeout: 3600
});
```

```
requirelib("filelib");
function run(job) {
  if (!requirelib("ffmpeg")) throw new Error("no ffmpeg on this node");
  job.progress(0.05);
  if (!ffmpeg.convert(job.args.input, job.args.output)) {
    throw new Error("conversion failed");
  }
  job.progress(1);
  return { output: job.args.output,
    bytes: filelib.filesize(job.args.output),
    node: job.node };
}
```

15.4 Map/reduce: count files per extension

```
requirelib("cluster");
var id = cluster.jobs.mapreduce({
  name: "Extensions in /photos",
  script: "user:/jobs/ext_count.agi",
  dataset: "/photos/**",
  partitionMax: 100
});
var rec = cluster.jobs.wait(id, 600);
sendJSONResp(JSON.stringify(rec.state.output)); // {"jpg": 812, ...}
```

```
// user:/jobs/ext_count.agi
function mapper(files, emit) {
  files.forEach(function (f) { emit(f.split(".").pop(), 1); });
}
function reducer(key, values) { return values.length; }
```

15.5 React to new files (event hook)

```
requirelib("cluster");
var hookId = cluster.on(["file.created"], "user:/hooks/on_new_file.agi");
```

```
// user:/hooks/on_new_file.agi
requirelib("cluster");
var ev = JSON.parse(postPara("event")); // {id, type, node, path, ...}
if (ev.path.match(/\.jpg$/i)) {
  cluster.emit("photos.uploaded", { path: ev.path }); // -> app.photos...
}
```

15.6 Inspect copies and set replicas

```
requirelib("cluster");
var rec = cluster.stat("cluster:/photos/a.jpg");
var where = rec.locations.map(function (l) {
    return l.nodeId + " (" + l.state + ")";
});
cluster.policy("/photos", 2); // admin: 2 copies for /photos
cluster.setReplicas("cluster:/photos/a.jpg", 3); // admin: this file, 3
sendResp(where.join(", "));
```

15.7 Submitting over HTTP

```
curl -b cookies.txt -X POST \
  https://node-a.example.com/system/cluster/jobs/submit \
  -d script=user:/jobs/hello.job.agi -d name=Hello \
  -d features=ffmpeg -d timeout=600 -d 'args={"x":1}'
```

List fields (`inputs`, `features`, `nodes`) accept a comma-separated list; `inputs` also accepts a JSON array.

16. Minimum dependencies and requirements

16.1 Software

Requirement	Detail
ArozOS binary	This branch or later on every member; the cluster is built in. No separate service, database or agent is installed.
Build toolchain	Go 1.25 (<code>go.mod</code>); <code>CGO_ENABLED=0</code> builds are supported.
Go modules used by the cluster	<code>github.com/gorilla/websocket</code> (BSD-2), <code>github.com/satori/go.uuid</code> (MIT), <code>github.com/boltdb/bolt</code> (MIT, through <code>mod/database</code>), <code>golang.org/x/sys</code> (BSD-3), <code>github.com/robertkrimen/otto</code> (MIT, AGI) and the Go standard library (Ed25519, SHA-256). All already in ArozOS; the cluster added no new module.
External tools	None required. <code>ffmpeg</code> , <code>docker</code> , <code>nvidia-smi</code> , <code>git</code> , <code>python3</code> , <code>node</code> only advertise optional capabilities that jobs may ask for.
Platforms	Every ArozOS target: Linux amd64/386/arm/arm64/mipsle/riscv64, macOS, Windows. Syscalls are build-tagged (<code>capability/diskusage_*.go</code>).

16.2 Cluster topology

Requirement	Detail
Minimum members	1 to create a cluster (and mount <code>cluster:/</code> once it has a volume); 2 for replication and failover; 3 for the site-diversity factor.
At least one reachable member	A member with an advertised URL must exist to issue join tokens and host tunnels. Other members may be NAT-only.
Unique node IDs	Each host needs its own <code>system/dev.uuid</code> (or <code>-uuid</code>).
Clock synchronisation	Clocks within plus or minus 5 minutes (signature window and assertion lifetime). Use NTP.
Network	HTTP(S) to the normal ArozOS port. No extra ports, multicast or VPN. WebSockets must pass through any proxy for tunnels. Cloudflare is supported.
At least one volume	Required before <code>cluster:/</code> appears.
Accounts	For jobs, the owner must exist on the executing node: configure an identity origin, or create accounts manually. Create the same permission groups on every node.
Disk	Room for the contributed volume plus the spool (<code>-tmp</code> folder) for files in transit; a volume is read-only below 5 % free.
HTTPS	Strongly recommended in production: the account directory carries password hashes.

16.3 Enabling and disabling

The cluster agent is available by default. Start with `-disable_cluster` to turn it off completely: no agent, no `cluster:/`, no cluster tabs, and `/cluster/acn/*` answers 404. The older mDNS Neighbourhood page is separate and governed by `-allow_mdns`. Relevant flags: `-uuid`, `-hostname`, `-port`, `-tmp`, `-disable_cluster`.

17. Limits and maximum capabilities

Area	Limit	Value
Cluster size	no hard cap; full-mesh heartbeats every 15 s make traffic grow with the square of the node count	designed for small clusters (households, labs, small offices)
Replicas	per folder policy	1 to 16
Replicas	per file	0 to 16 (0 = policy)
File size	no fixed maximum; bound by the target volume	transferred in 4 MiB chunks
Chunk	default / maximum	4 MiB / 8 MiB
Upload session	idle expiry	30 minutes
Signed request body	maximum	64 MB
Tunnel frame	maximum	16 MB
Clock skew	tolerated	plus or minus 5 minutes
Jobs per node	concurrent	<code>runtime.NumCPU()</code>
Job timeout	default	3600 s
Job attempts	default	3
Job log	kept	last 200 lines
Job records	retention after finishing	72 hours
Map/reduce	partition size default	50 files
Map/reduce	grouped map output	8 MB
Scheduling pass	interval	3 s
Replication	tasks in flight	4 per node, 16 total
Replication	retries	5, then pause 1 hour
Verification	nightly budget	1 GB per node
Custom event payload	maximum	64 KB
Hook scripts	concurrent per node	4
Join token	default lifetime	24 hours
User assertion	lifetime	5 minutes

What a cluster can do at its best:

- Present one `cluster:/` namespace, readable and writable from every member and every app, with files transparently fetched from whichever node holds them.
- Keep up to 16 verified copies of each file on distinct nodes, repair and re-verify them automatically, spread second copies across sites, and move data off retiring disks without downtime.
- Survive the loss of any node, including the leader and the identity origin, with two-node failover and no quorum.
- Run AGI jobs on any architecture, placed by capability, data locality and load, with progress, logs, cancellation, retries and explanations; run a job on every node at once; and run map/reduce where the data lives.
- Connect nodes across the Internet, behind NAT or Cloudflare, over the normal HTTPS port.

18. Security model

Concern	Mechanism
Node authentication	Ed25519 signature over method, target, cluster, node, timestamp, nonce and body hash
Replay	per-request nonce, spent only after full verification; 5-minute clock window
Cross-cluster confusion	<code>X-Aroz-Cluster</code> must match
Relayed requests	signature covers the original path; both relay and target verify
Joining	one-time secret in the token, stored as SHA-256, expiring and revocable
Passwords	only SHA-512 hashes are forwarded; constant-time comparison at the origin
Acting for a user across nodes	signed, short-lived <code>X-Aroz-User</code> assertions
Browser endpoints	permission router; admin-only for cluster control
Jobs	run as the owner through the AGI sandbox, with the owner's permissions on the executing node
Data integrity	per-chunk and whole-file SHA-256, verified copies, nightly re-checksums
State isolation	<code>system/c1uster.db</code> , never <code>ao.db</code> ; cluster tables wiped on leave

Operational advice: run members behind HTTPS; protect `system/c1uster/node.key` like a private key; revoke unused join tokens; keep clocks synchronised; and do not clone a system folder without deleting `dev.uuid` and `c1uster/`.

19. Operations and administration

Three System Settings tabs in the **Cluster** group (localized in en-us, zh-tw, zh-hk, zh-cn, ja-jp, ko-kr):

Tab	Page	Contents
Cluster Settings	<code>SystemA0/ccluster/ccluster.html</code>	node name and URL, create/join/leave, join tokens, identity origin, volumes and the nearly-full toggle, replica policies, maintenance and draining, scheduling weights
Cluster Info	<code>SystemA0/ccluster/cclusterinfo.html</code>	reachability, platform, health, cluster summary, namespace state, node table with probes, replication tools, score preview, latency matrix, job explainer
Cluster Jobs	<code>SystemA0/ccluster/jobs.html</code>	submit, follow and cancel jobs (users see their own)

Typical setup

1. On node A (reachable URL): Cluster Settings > set the advertised URL > Create cluster.
2. Add a volume, for example `user:/ccluster`. `ccluster:/` now appears.
3. Generate a join token, paste it on node B (and C, ...).
4. Choose an identity origin and create the same permission groups on each node.
5. Set replica policies, for example `/photos = 2`.
6. Watch Cluster Info for node states, replication health and scores.

Maintenance

- Set a node to `MAINTENANCE` before rebooting it, or `DRAINING` to move its copies away while it still serves reads.
- Evacuate a volume before removing a disk.
- Use "Verify this node" after suspected disk problems.
- For a live two-node test on one machine, see TASKS.md section 0.5: two isolated copies of `system/`, separate ports, `-uuid` and `-hostname`, `-allow_mdns=false` `-allow_ssd=false`.

20. Developer guide: extending the cluster

20.1 Repository rules that apply

1. Log only through `logger.PrintAndLog("Cluster", message, err)`.
2. Every package under `src/mod/` ships tests; new exported functions are covered (table-driven, `t.TempDir()`, `t.Fatalf`).
3. New dependencies must be MIT, BSD, Apache-2.0, MPL-2.0 or ISC.
4. Browser endpoints go through `prout.NewModuleRouter`; node-to-node endpoints through `acn.Server.HandleFunc` (signed, mounted under `/cluster/acn/`).
5. Stay portable: no hardcoded OS paths, no `exec.Command` in shared code, build tags for OS-specific code.
6. No emoji in code or UI; use Semantic UI icons or SVG.
7. Add user-facing strings and server messages to `src/web/SystemA0/locale/cluster.json` (`CL.t` for labels, `CL.tr` for server messages via `msg/<text>` and `msgp/<template>` entries).

20.2 Adding a replicated feature

1. Pick a record kind: reuse `metadata.KindSetting` for configuration, or add a kind with a `Version` field that merges last-writer-wins.
2. Write with `Submit(kind, record)`; never write the disk directly under a lock.
3. Serialise decisions on the leader (`IsLeader()`); make tasks ephemeral so a new leader can re-plan.
4. Use `scheduling` for any "which node" choice so weights and explanations stay consistent.
5. Publish events for anything scripts may want to react to.
6. Register any new table with `membership.RegisterClusterTable`.

20.3 Testing multi-node logic

Follow `identity/identity_test.go`: create N `membership.NewManager` instances with `t.TempDir()` databases, serve each with `httptest.NewServer(m.ACNHandler())`, set URLs with `m.UpdateConfig`, create a cluster on one, `NewJoinToken` + `JoinCluster` the others, and poll with `waitFor(...)` (5 s deadline); never sleep fixed durations. Timing constants are variables so tests can shorten them in `init()`. Close managers in `t.Cleanup`.

```
cd src
go build ./... && go vet ./mod/cluster/...
go test -timeout 300s -count=3 ./mod/cluster/...
sh ../scripts/check-conventions.sh --diff origin/master
```

21. Known constraints and pitfalls

- **No `-race` for Bolt-backed packages.** `boltdb/bolt` panics under `checkptr`; only `acn` can run with `-race`.
- **Two ArozOS instances cannot share `src/`** (Bolt lock on `ao.db`).
- **Page scripts must not declare `var status`**: it shadows `window.status`. Use names such as `clusterStatus`.
- **`go mod tidy` pulls in modules imported by user files** under `src/files/`; revert unrelated `go.mod` additions.
- **CRLF files.** If an edit produces a whole-file diff, normalise with `sed -i 's/\r$//' <file>`.
- **Windows temp-dir cleanup** fails if a test leaves a Bolt DB open.
- **Relative paths in jobs** do not resolve; use `cluster:/` or `user:/`.
- **Groups are per node.** A mirrored account only gets groups that exist on that node.
- **Eventual consistency.** Two writers of the same path on different nodes resolve last-writer-wins; applications needing stronger guarantees should coordinate through a job or the leader.
- **Map output lives in the job record**, so keep emitted values small (8 MB cap).

22. Appendix A: endpoint reference

A.1 Signed node-to-node endpoints (/cluster/acn/...)

Group	Endpoints
Transport	GET hello (unsigned probe), POST ping, GET tunnel, * relay/{node}/..., GET latency
Membership	POST join (join token), POST heartbeat, GET members, POST members/sync, POST leave, POST evict
Identity	POST auth/verify, GET auth/directory, POST auth/setpassword
Metadata	meta/lease, meta/append, meta/submit, meta/log, meta/snapshot, meta/stat
Storage	store/place, store/begin, store/chunk, store/commit, store/abort, store/read, store/stat, store/list, store/checksum, store/mkdir, store/rename, store/delete
Replication	repl/pull, repl/lease, repl/done, repl/plan
Events	events/publish
Jobs	jobs/run, jobs/done, jobs/submit

A.2 Browser endpoints (/system/cluster/...)

Group	Endpoints	Access
Agent	status, create, join, leave, config, testurl, nodes, capabilities, node/remove, node/state, node/probe	admin
Tokens	token/new, token/list, token/revoke	admin
Identity	identity/status, identity/origin, identity/sync	admin
Metadata	meta/status, meta/ls, meta/stat, meta/policy/list, meta/policy/set	admin
Storage	storage/status, storage/volume/add, storage/volume/remove, storage/volume/readonly, storage/volume/evacuate, storage/volume/evacuate/cancel, storage/volume/evacuate/status, storage/rescan, storage/autoreadonly	admin
Replication	repl/status, repl/plan, repl/verify	admin
Scheduling	sched/status, sched/weights, sched/explain?job=<id>	admin
Jobs	jobs/submit, jobs/status, jobs/get, jobs/cancel	user (own jobs)
Events	events/ws, events/hooks	user
Neighbourhood (mDNS)	scan, record, wol	user, only with -allow_mdns

23. Appendix B: tunables and constants

Constant	Value	Package
HeartbeatInterval	15 s	membership
OnlineWindow / OfflineWindow	45 s / 3 min	membership
TombstoneTTL	7 days	membership
LatencyMatrixTTL	2 min	membership
MaxClockSkew	5 min	acn
MaxSignedBody	64 MB	acn
TunnelPingInterval / TunnelReadTimeout	30 s / 90 s	acn
TunnelMaxFrame	16 MB	acn
DefaultRequestTimeout	60 s	acn
DefaultAssertionTTL	5 min	identity
DefaultSyncInterval	5 min	identity
Leader lease / renew	30 s / 10 s	metadata
FlushInterval	50 ms	metadata
ChunkSize / MaxChunkSize	4 MiB / 8 MiB	storage
SessionTTL	30 min	storage
Refresh / reconcile	60 s / 30 min	storage
Low-space guard	5 % on, 7 % off	storage
Plan interval	60 s	replication
In flight per node / total / workers	4 / 16 / 4	replication
Max attempts	5	replication
Offline-to-stale	10 min	replication
Schedule interval	3 s	jobs
Task lease / renew	30 s / 10 s	jobs
Max parallel	NumCPU	jobs
Retention	72 h	jobs
Default timeout / attempts	3600 s / 3	jobs
Log lines	200	jobs
Batch interval / dedup window	200 ms / 10 min	events
Concurrent hook runs	4	events

24. Appendix C: glossary

Term	Meaning
Node	one ArozOS installation
Member	a node that is in the cluster
Leader / master node	the member holding the metadata lease; serialises placement, replication, scheduling and shared nightly maintenance
Identity origin	the member that owns the account table
Volume	a folder a node contributes to the cluster namespace
Record	one entry in the metadata store
Copy / location	one physical copy of a file on a volume
Policy	desired replica count for a top-level folder
ACN	ArozOS Cluster Node protocol
ACMS	ArozOS Cluster Metadata Store
AID	ArozOS Identity
AGI	ArOZ Online JavaScript Gateway Interface (the server-side JavaScript runtime)
Tunnel	a WebSocket a NAT-only node keeps open to a reachable member
Relay	forwarding a request to a tunnelled node through its tunnel host